

Manual Técnico

Puente (edificios A-B) de la Facultad de Ingeniería, UNAM



**Proyecto Final
Elaborado por:**

320249538

Grupo: 05

Semestre: 2026-2

**Fecha de entrega:
26/05/2026**

Nota sobre el trabajo en equipo: El proyecto fue desarrollado en colaboración con 423095203 durante el laboratorio de la asignatura. La presente documentación refleja específicamente mis contribuciones individuales, conforme a los lineamientos de entrega del curso de teoría. Las secciones de arquitectura, código fuente y descripción general de la escena documentan el proyecto en su totalidad, indicando explícitamente en cada caso qué componentes fueron desarrollados.

1. Objetivo del proyecto

1.1 Objetivo general

Desarrollar e integrar un entorno virtual tridimensional interactivo en tiempo real que reproduzca con fidelidad arquitectónica el puente que conecta los edificios A y B del primer piso de la Facultad de Ingeniería de la UNAM, aplicando los conocimientos teóricos adquiridos durante el curso.

El entorno simula la disposición habitual del puente durante ferias académicas y eventos de reclutamiento, incorporando seis stands representativos de empresas patrocinadoras y asociaciones estudiantiles, así como un sistema completo de iluminación dinámica, animaciones y navegación libre mediante cámara sintética.

1.2 Objetivos específicos

- Implementación gráfica correcta: Aplicación del modelo de iluminación de Phong (componentes ambiental, difusa y especular) sobre la totalidad de los modelos cargados, integrando luces direccionales, puntuales y reflectores cónicos (spotlights).
- Modelado tridimensional: Construcción de geometría limpia, con normales correctamente orientadas, coordenadas UV consistentes y texturas que reflejen materiales reales (concreto, metal, cristal, papel impreso).
- Cámara sintética funcional: Implementación de una cámara en primera persona capaz de explorar libremente la escena mediante teclado y ratón.
- Siete animaciones: Cada animación debe poseer contexto narrativo dentro del entorno, coherencia visual con la escena, y justificación matemática en el caso de las animaciones complejas.
- Funcionamiento estable del sistema gráfico: Ejecución sin errores, sin pérdidas de memoria detectables y con rendimiento aceptable en hardware de gama media.
- Optimización del peso del proyecto: Conjunto de modelos y texturas inferior a 100 MB, sin archivos comprimidos en el repositorio y sin rutas absolutas en el código fuente.

1.2 Contribuciones individuales

Dentro del proyecto desarrollado en equipo, mis aportaciones específicas fueron las siguientes:

- **Modelado 3D:**

Construcción completa del modelo del puente en Blender, incluyendo el piso, los pilares cilíndricos de soporte, el techo, los barandales perimetrales y las bardas laterales.

Texturizado del puente mediante UV mapping y aplicación de materiales de concreto y acero en Blender.

- **Sistema de iluminación:**

Implementación de las 24 luces tipo spotlight distribuidas sobre los seis stands (cuatro por cada stand). Configuración individual de su posición, dirección, ángulos de apertura (interior y exterior), atenuación, color e intensidad diferenciada según el modo día/noche activo.

- **Sistema de skyboxes:**

Diseño e implementación del sistema dual de skyboxes (interior + exterior) con cuatro cubemaps independientes (interior día, interior noche, exterior día, exterior noche), cambio dinámico mediante la tecla L, y configuración de los shaders correspondientes para que el skybox interior se ajuste a las dimensiones reales del puente extraídas de su geometría.

- **Animaciones (4 de las 7):**

1. Animación del visitante caminante: Mediante KeyFrames con interpolación lineal por pasos (10 fotogramas clave). Implementación de lógica de doble estado para alternar entre la animación de caminata y una pose fija de observación frente a los stands.
2. Animación del presentador del stand: Mediante una jerarquía de cuatro partes (cuerpo, cabeza, antebrazo, mano) con rotaciones generadas por funciones sinusoidales independientes para cada articulación.
3. Animación del brazo robótico: Mediante KeyFrames jerárquicos con cinco niveles de transformación encadenada (base fija → pedestal → hombro → codo → garra) y nueve fotogramas clave que describen un ciclo completo de manipulación.
4. Animación de los folletos volando: Combina trayectorias trigonométricas (sin/cos) con una lógica de dos fases (vuelo y reposo) implementada mediante "fmod()", y una pila estática de folletos acumulados sobre la mesa del stand de P&G.

2. Alcance y limitantes

2.1 Alcance del proyecto

- Navegación libre del usuario: El usuario controla una cámara sintética en primera persona y puede recorrer cualquier punto del entorno sin recorridos predefinidos ni restricciones de trayectoria.
- Alternancia entre dos condiciones de iluminación: El usuario puede cambiar dinámicamente entre un esquema diurno y uno nocturno mediante una sola interacción de teclado, observando cómo el comportamiento de las luces de los stands, las lámparas de techo y los cuerpos celestes responde al cambio.
- Activación selectiva de animaciones: El usuario puede activar o desactivar individualmente las animaciones interactivas mediante teclas dedicadas, lo que permite enfocar la atención en elementos específicos del entorno durante su exploración.
- Distribución y despliegue local: El proyecto se entrega como una solución compilable en Visual Studio, alojada en un repositorio público de GitHub. Cualquiera puede clonarla, compilarla y ejecutarla en su propio equipo sin configuración adicional.

2.2 Limitantes

- No se implementó detección de colisiones. La cámara puede atravesar paredes, pilares, stands y cualquier modelo de la escena. El visitante animado tampoco percibe obstáculos y recorre su trayectoria predefinida sin importar lo que se interponga.
- No se implementaron sombras dinámicas. El modelo de iluminación de Phong aplicado no contempla shadow mapping ni oclusión ambiental, por lo que ningún objeto proyecta sombras sobre otro. En escenas nocturnas esta limitación es perceptible, pero su corrección implicaba un pipeline de renderizado más complejo.
- La cámara opera únicamente en primera persona. No se incluyeron vistas alternativas ni un sistema de cambio entre cámaras múltiples. La elección de una cámara única en primera persona se alinea con el caso de uso principal del proyecto: simular la perspectiva de un visitante real recorriendo el puente.
- El peso total del proyecto se mantuvo por debajo de los 100 MB. Este límite, exigido por el requerimiento del curso, condicionó la resolución máxima de las texturas y la densidad poligonal de los modelos. Algunos elementos podrían beneficiarse de texturas en mayor resolución, pero la decisión fue priorizar el cumplimiento estricto del requerimiento sobre la fidelidad visual.

3. Estado del arte

3.1 Sistemas de campus virtual

Los sistemas de campus virtual son aplicaciones gráficas que reproducen, mediante geometría tridimensional, las instalaciones físicas de una institución educativa con el fin de facilitar la navegación, la planeación de eventos o la difusión institucional hacia estudiantes prospectos.

Uno de los trabajos pioneros en esta línea es el desarrollado por la Anhui Normal University de China, donde se construyó un sistema virtual completo del campus combinando SketchUp para el modelado tridimensional y OpenGL para el renderizado en tiempo real. El sistema demostró ser ejecutable en hardware de PC convencional sin requerimientos especializados, lo que validó la viabilidad de este tipo de aplicaciones académicas.

Más recientemente, un proyecto realizado para modelar el campus de una universidad utilizando Blender y Three.js exploró el uso de geometría calculada de forma proporcional al espacio físico real, asegurando que las dimensiones del modelo virtual reflejaran fielmente las medidas arquitectónicas del campus.

En el ámbito web, la University of Mauritius desarrolló un tour virtual completo de su campus utilizando WebGL, una variante de OpenGL adaptada para navegadores. El propósito explícito fue ayudar a estudiantes nacionales e internacionales a familiarizarse con el campus antes de llegar físicamente. Este caso de uso es exactamente el mismo que motiva el presente proyecto: permitir que estudiantes, empresas patrocinadoras y visitantes externos puedan explorar el espacio del puente y comprender cómo se distribuyen los stands durante eventos académicos, sin necesidad de presencia física.

A nivel comercial, empresas como THE FUTURE 3D ofrecen servicios profesionales de tours virtuales de campus universitarios mediante tecnología Matterport y fotografía aérea con drones, capturando edificios, aulas, salones, residencias y espacios al aire libre en detalle fotorrealista.

3.2 Modelo de iluminación de Phong

El sistema de iluminación implementado en el proyecto se basa en el modelo de reflexión propuesto por Bui Tuong Phong en su tesis doctoral de 1975 en la Universidad de Utah. El modelo de reflexión de Phong es un modelo empírico de iluminación local que calcula la apariencia de una superficie en un punto determinado, considerado un radical en su momento pero convertido desde entonces en el método base de sombreado para numerosas aplicaciones de renderizado.

El modelo de Phong se compone de tres componentes principales: iluminación ambiental, difusa y especular. Esta descomposición es exactamente la utilizada en el shader "lighting.frag" del presente proyecto, donde cada fragmento calcula su color final sumando las contribuciones de los tres componentes para cada fuente de luz presente en la escena.

3.3 Bibliotecas gráficas utilizadas

OpenGL: API gráfica multiplataforma para renderizado 2D y 3D con aceleración por hardware.

GLFW: Biblioteca de gestión de ventanas y eventos de entrada, optimizada para aplicaciones OpenGL.

GLEW: Cargador de extensiones de OpenGL que permite acceder a las funciones modernas del API.

GLM (OpenGL Mathematics): Biblioteca de matemática vectorial y matricial diseñada específicamente para programación gráfica, con sintaxis compatible con GLSL.

Assimp: Biblioteca de importación de modelos tridimensionales que soporta más de cuarenta formatos distintos, incluyendo OBJ y FBX.

SOIL2 : Biblioteca de carga de texturas optimizada para OpenGL.

3.4 Posicionamiento del presente proyecto

Frente al estado del arte descrito, el presente proyecto se ubica en la intersección de tres líneas: los sistemas académicos de campus virtual, las simulaciones interactivas para planeación de eventos institucionales, y los entornos pedagógicos para la enseñanza de computación gráfica. Las características que lo diferencian son:

- 1.Especificidad del caso de uso: A diferencia de los tours virtuales generales, el proyecto está centrado en un espacio puntual del campus (el puente entre edificios A y B) y en un caso de uso muy específico (la simulación de ferias y eventos con stands), no en una representación exhaustiva de la facultad completa.
- 2.Combinación de técnicas de animación: El proyecto integra simultáneamente animación esquelética, animación por KeyFrames con interpolación lineal, animación jerárquica y animación mediante funciones trigonométricas.
- 3.Iluminación dual día/noche: A diferencia de la mayoría de los tours virtuales académicos, que utilizan una sola condición de iluminación fija, el proyecto incorpora un sistema dinámico que permite alternar entre dos esquemas completos de iluminación.

4. Tecnologías utilizadas y justificación

4.1 Lenguaje y motor gráfico

C++ (Visual Studio 2022)

Se eligió C++ como lenguaje principal por su acceso directo a la memoria del sistema, su eficiencia en operaciones matemáticas intensivas y su compatibilidad nativa con las bibliotecas gráficas modernas. A diferencia de lenguajes de mayor nivel como Python o JavaScript, C++ permite controlar de forma explícita la asignación de buffers en GPU, la gestión de uniformes en los shaders y la liberación de recursos, lo cual es indispensable en aplicaciones gráficas en tiempo real donde el rendimiento por fotograma es crítico.

Además durante todo el curso se trabajó de esta manera.

OpenGL

OpenGL fue seleccionado como API gráfica por tres razones principales:

1. Estándar académico: Es la API utilizada de forma consistente en las prácticas de laboratorio del curso, lo que aseguró que el equipo dominara su sintaxis y patrones de uso.
2. Pipeline programable: A diferencia del pipeline fijo de versiones anteriores, obliga a programar explícitamente los vertex shaders y fragment shaders, lo cual refuerza la comprensión real del proceso de renderizado.
3. Multiplataforma: OpenGL cuenta con documentación oficial extensa y una comunidad activa, lo que facilita la resolución de problemas durante el desarrollo.

4.2 Bibliotecas de soporte

GLFW

Encargada de la creación de la ventana, la gestión del contexto de OpenGL y el manejo de eventos de teclado y ratón. Se prefirió sobre alternativas como GLUT o SDL porque GLFW está específicamente diseñada para OpenGL moderno, ofrece soporte directo para los callbacks utilizados en la cámara sintética del proyecto.

GLEW

Permite cargar dinámicamente las extensiones de OpenGL durante la ejecución, evitando tener que vincular manualmente cada función. Es la solución estándar para acceder a las funciones modernas del API en sistemas Windows, donde los drivers no exponen todas las funciones de OpenGL de forma directa.

GLM

Biblioteca de matemática vectorial y matricial con sintaxis intencionalmente similar a GLSL. Esta similitud reduce la fricción al pasar datos entre el código C++ del CPU y los shaders del GPU.

GLM se utilizó en el proyecto para construir matrices de transformación (translación, rotación, escala) en las animaciones jerárquicas del brazo robótico y el presentador, calcular las matrices de vista y proyección de la cámara y aplicar rotaciones sobre pivotes, técnica utilizada en todas las animaciones jerárquicas que desarrollé.

Assimp

Encargada de la carga de modelos tridimensionales en formato OBJ y FBX. Permite cargar archivos OBJ con sus respectivos .mtl y texturas asociadas, procesa las jerarquías de huesos y los canales de animación de los modelos FBX, soporta más de varios formatos distintos, lo que permite trabajar con modelos descargados de fuentes diversas (CGTrader, Free3D, Mixamo) sin necesidad de conversión manual.

SOIL2

Carga de texturas en formatos JPG y PNG directamente a memoria de video. Se eligió frente a alternativas como stb_image por su API más concisa para la carga de cubemaps, función indispensable para el sistema de skyboxes que desarrollé. SOIL2 también gestiona automáticamente el mipmapping, mejorando el filtrado de texturas vistas a distancia.

4.3 Software de modelado

Blender 5.1

Se utilizó para construir desde cero la geometría del puente, incluyendo piso, pilares cilíndricos, techo, barandales y bardas. Sirvió para aplicar el UV mapping y los materiales con texturas de concreto y acero al puente. Se utilizó para definir y extraer los pivotes de rotación utilizados en las animaciones jerárquicas. La capacidad de Blender de mostrar las coordenadas locales del cursor 3D fue clave para obtener los puntos exactos sobre los que se aplican las rotaciones del brazo robótico y del presentador. Sirvió para exportar los modelos en formato OBJ con materiales y normales, y en formato FBX con las armaduras y animaciones horneadas. La elección de Blender frente a alternativas comerciales como Maya o 3ds Max responde a tres factores: es gratuito y accesible para estudiantes, tiene exportadores robustos para OBJ y FBX compatibles con OpenGL, y cuenta con una comunidad amplia que facilita la solución de problemas técnicos durante el modelado. Además de su curva de aprendizaje que es menor en comparación con las otras.

4.4 Herramienta de colaboración y control de versiones

GitHub

Se utilizó para mantener el historial completo de cambios del proyecto, permitiendo revertir en caso de errores. Se pudo trabajar en paralelo mediante ramas independientes para cada integrante realizando merges cuando los cambios estaban estables, sirvió como mecanismo de entrega del proyecto.

5. Diagrama de Gantt

El presente diagrama de Gantt refleja mi contribución individual al proyecto desarrollado en colaboración con mi compañero. El cronograma se organiza en cinco periodos de trabajo que abarcan desde la configuración inicial del repositorio el 19 de abril de 2026 hasta la entrega final de la documentación de teoría el 26 de mayo de 2026, e incluye dos entregas: la del proyecto para laboratorio (13 de mayo) y la del proyecto para teoría (26 de mayo). Las fechas y la asignación de tareas se sustentan en mi historial real de commits en GitHub bajo el usuario SamuelMV09, conforme a la evidencia disponible en el repositorio público del proyecto.

Diagrama de Gantt por 320249538

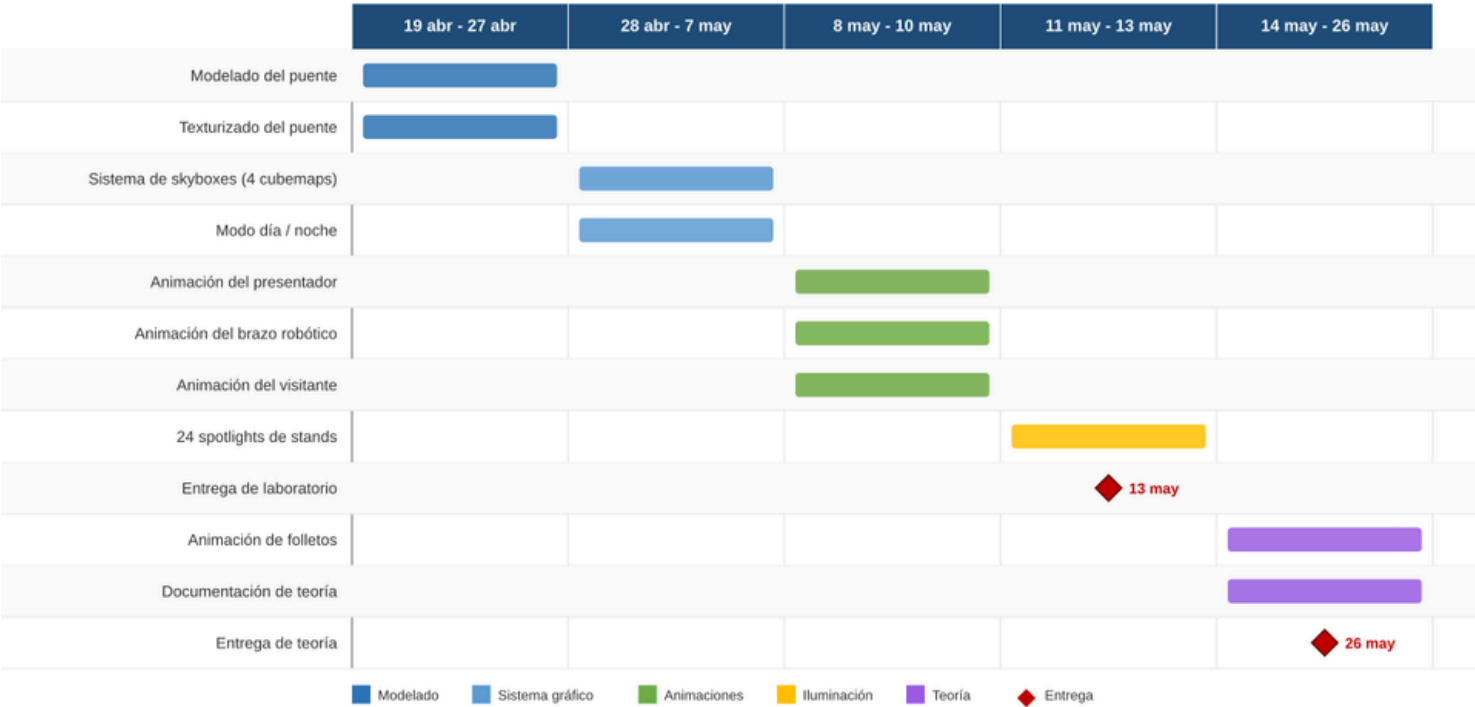
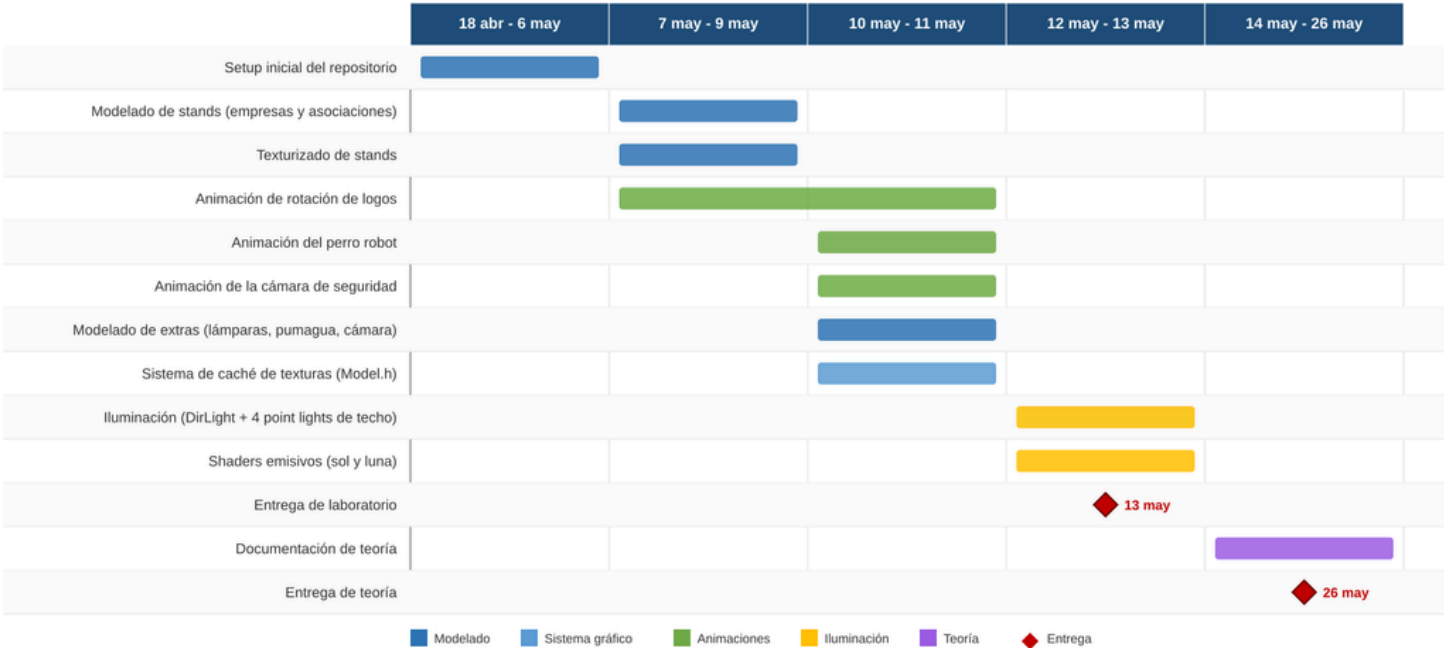
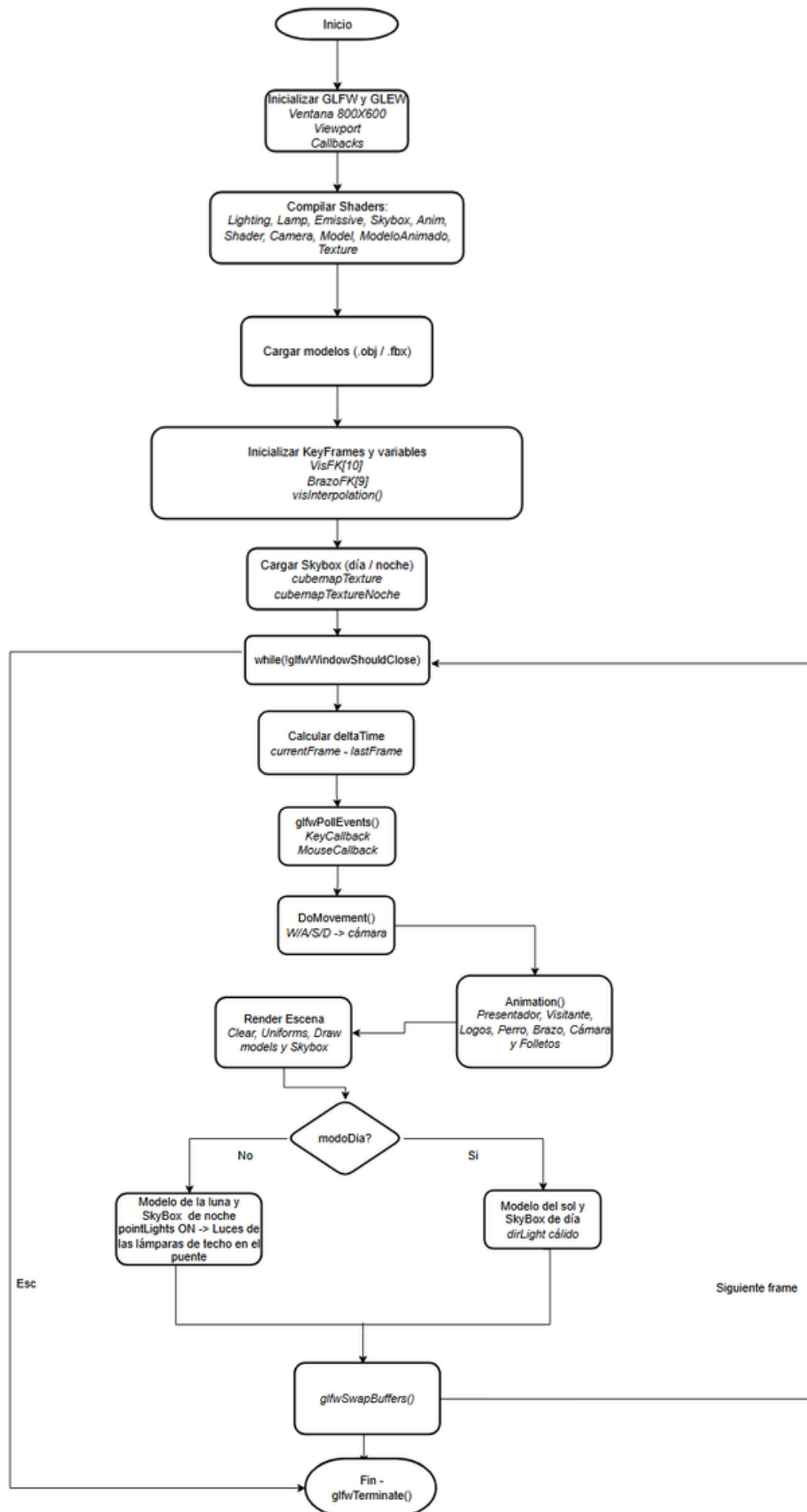


Diagrama de Gantt por 423095203

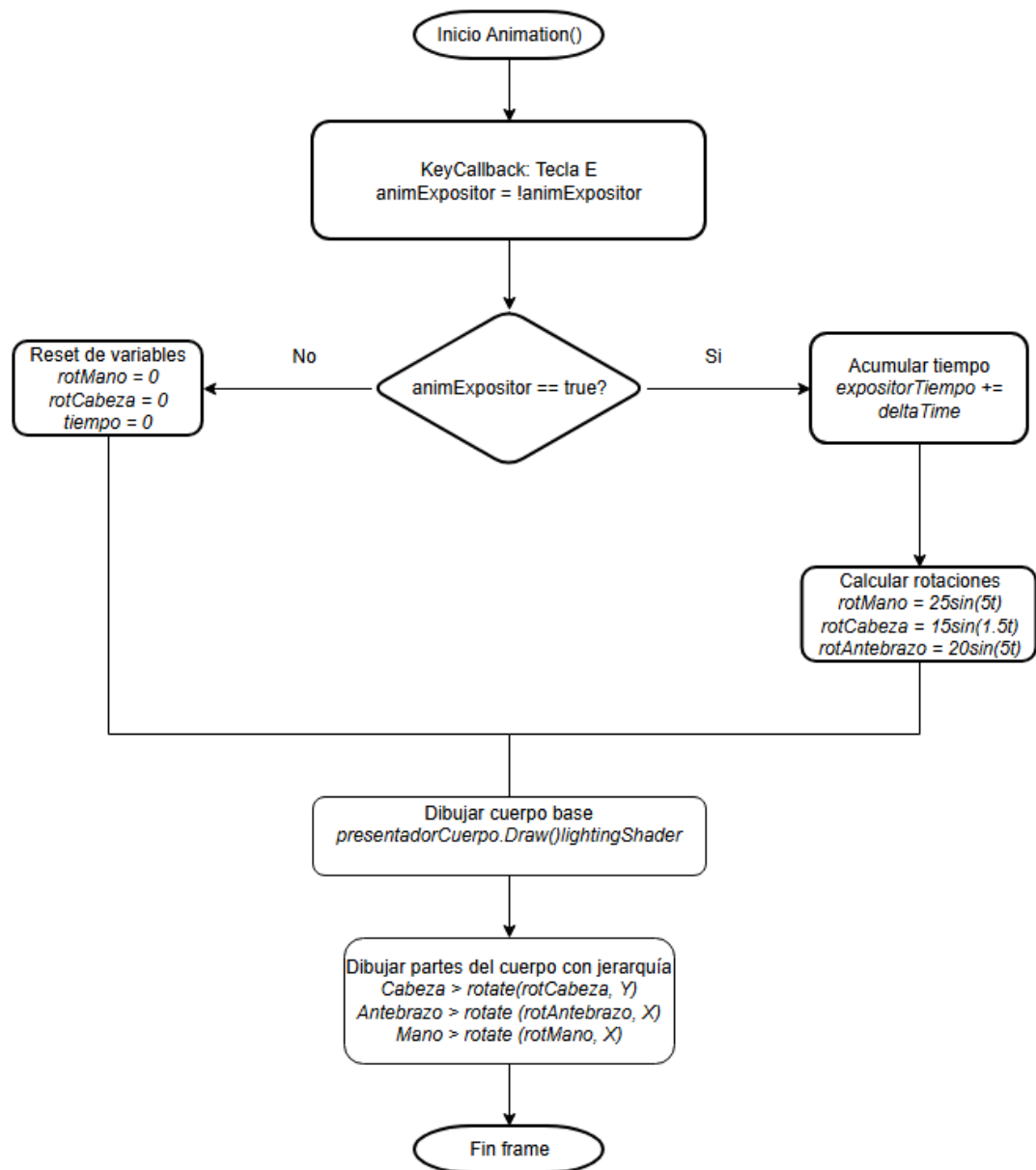


6. Diagramas de flujo del sistema

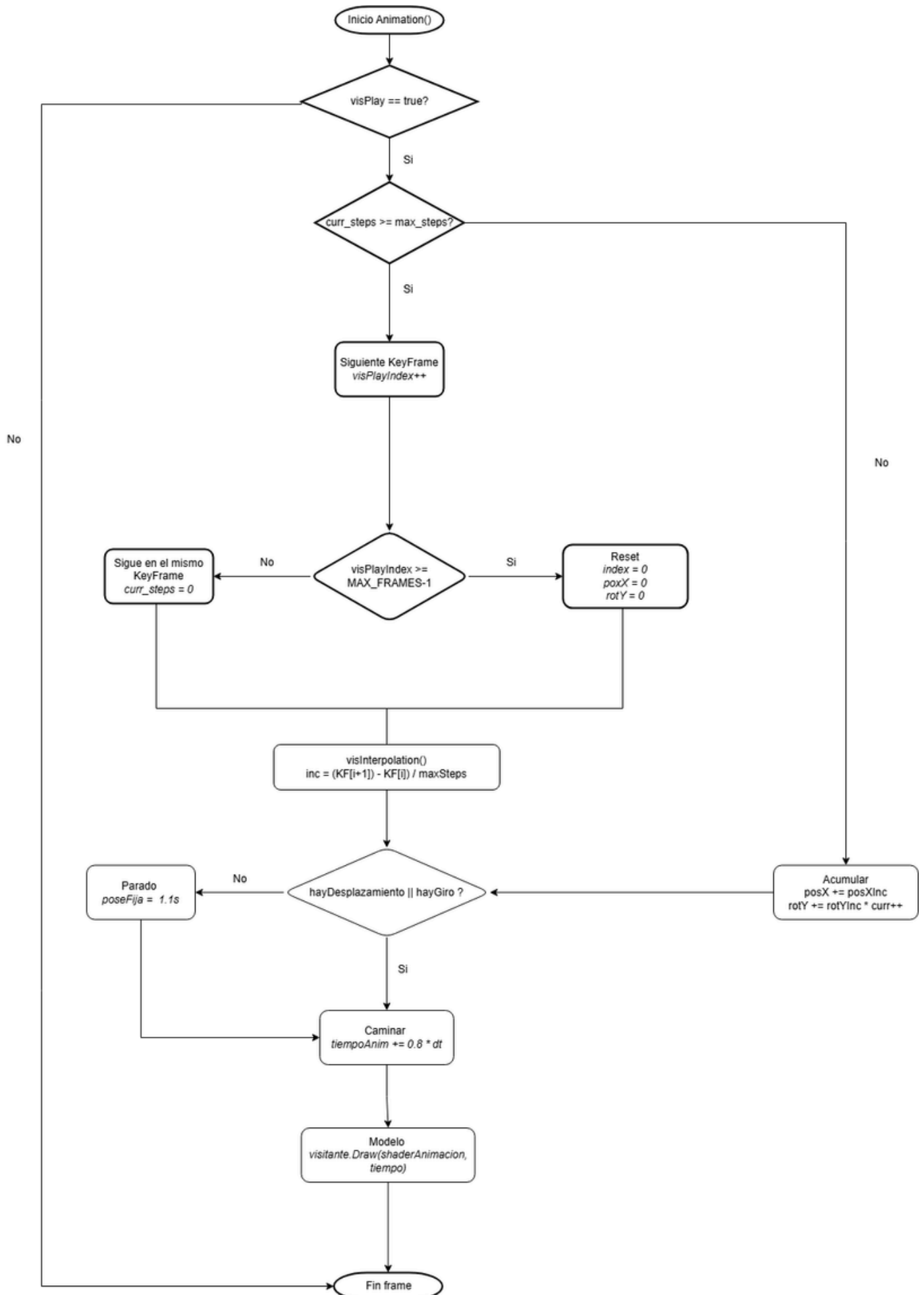
6.1 Diagrama general del sistema



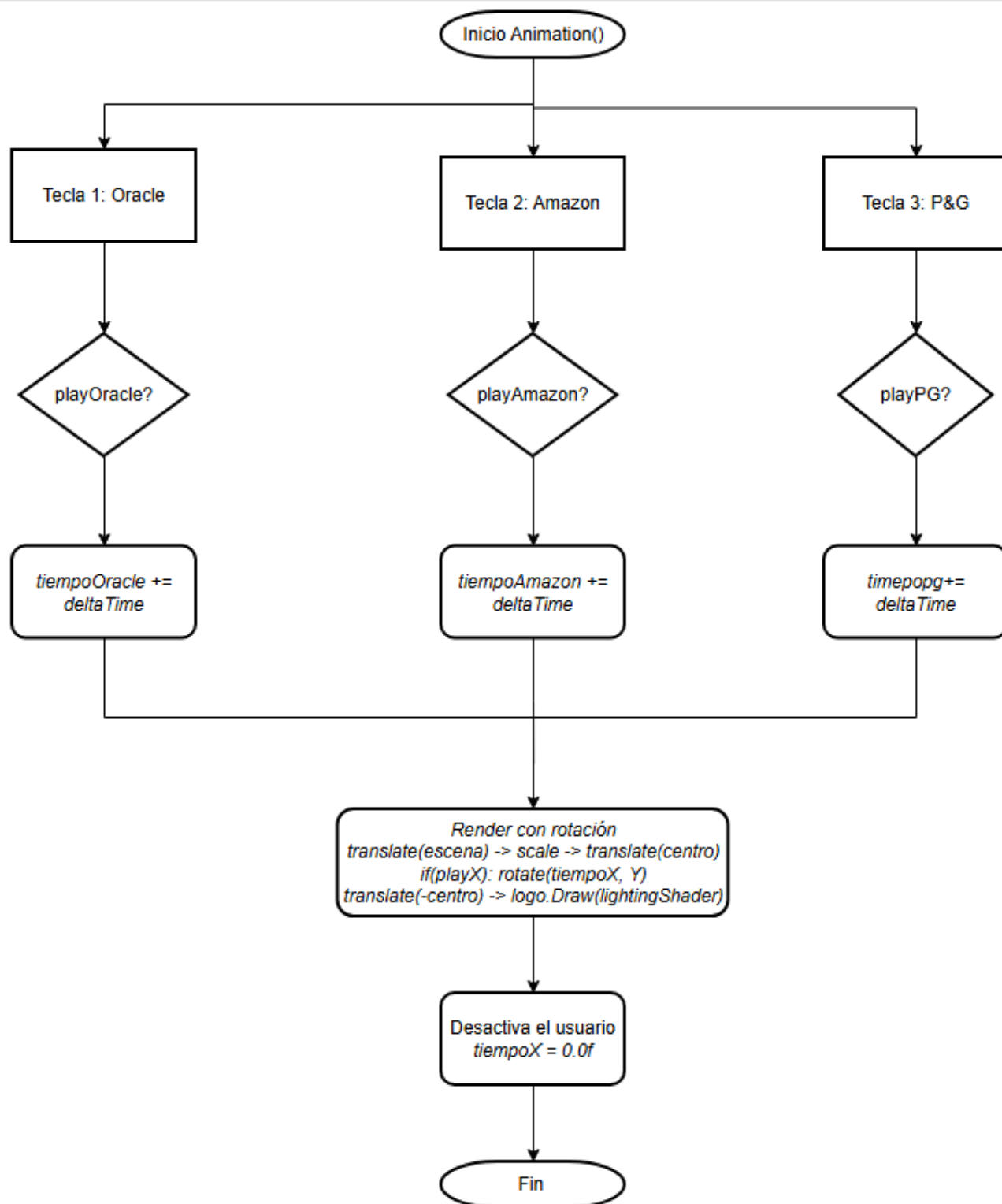
6.2 Diagrama de la animación del presentador



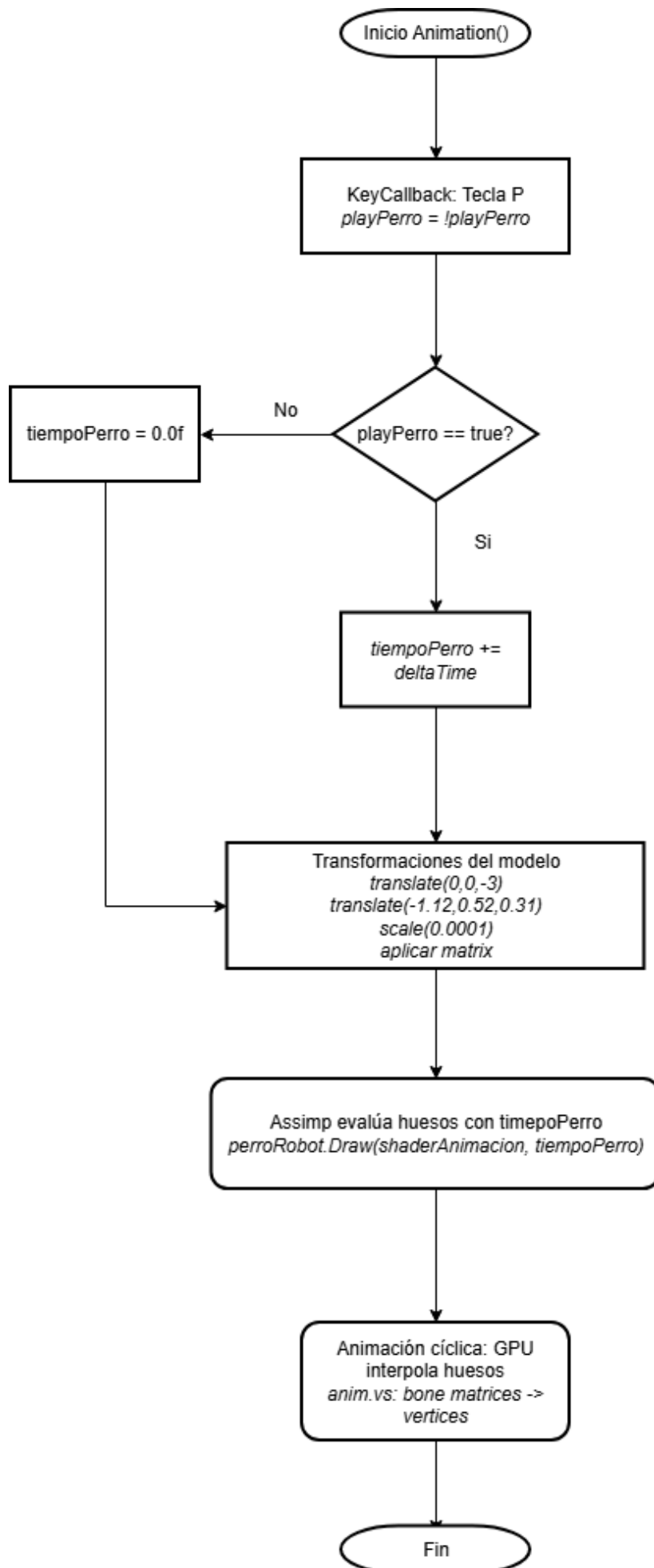
6.3 Diagrama de la animación del visitante



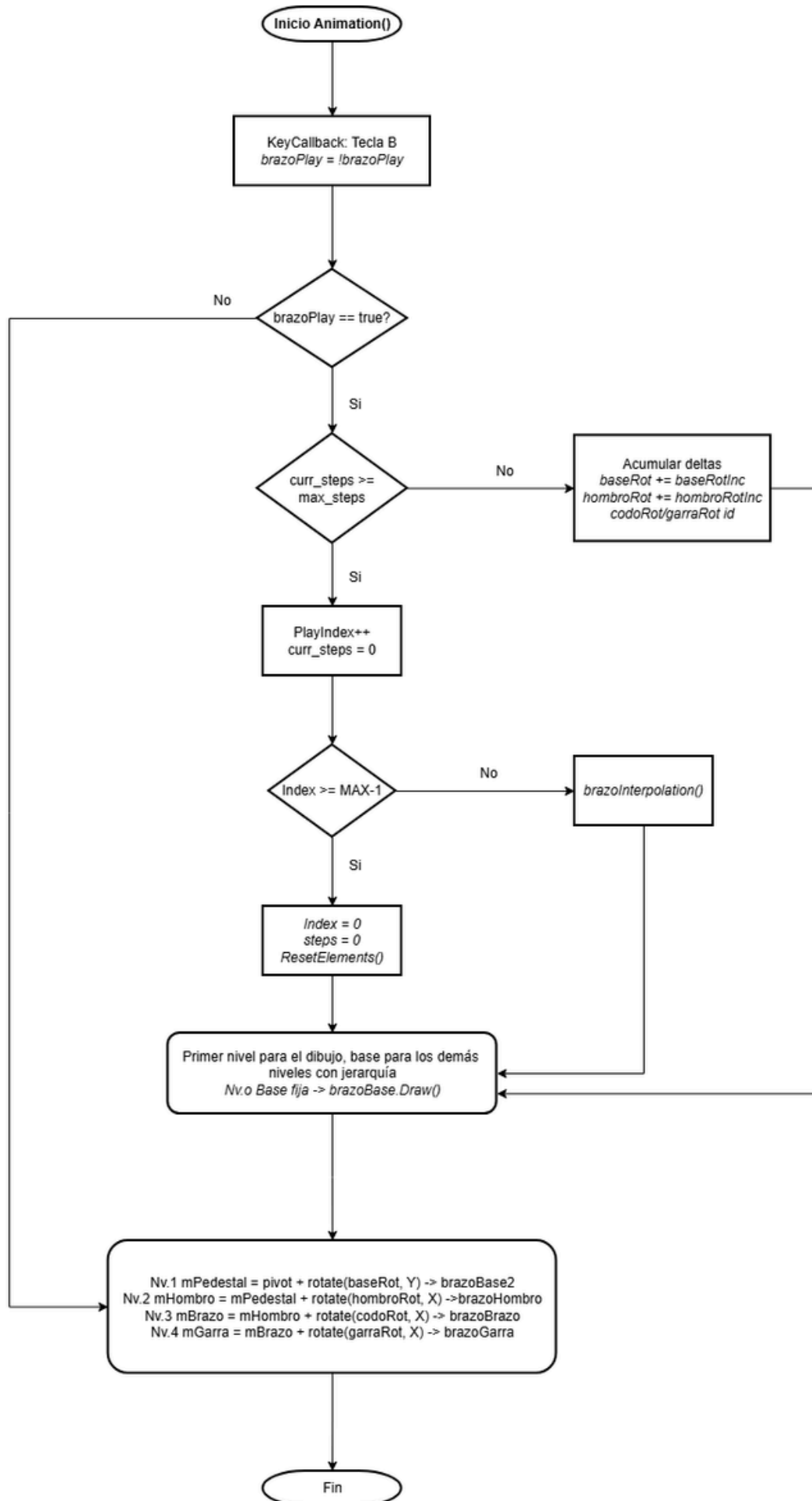
6.4 Diagrama de la animación de los logos



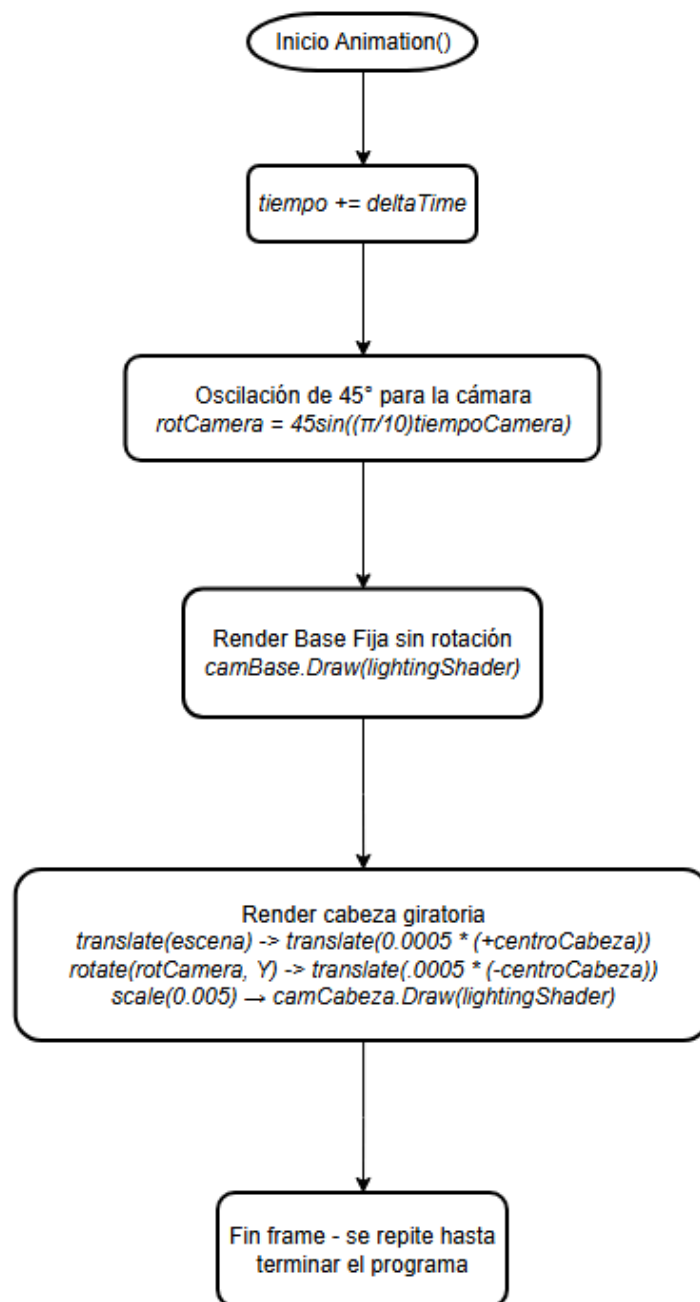
6.5 Diagrama de la animación del perro robot



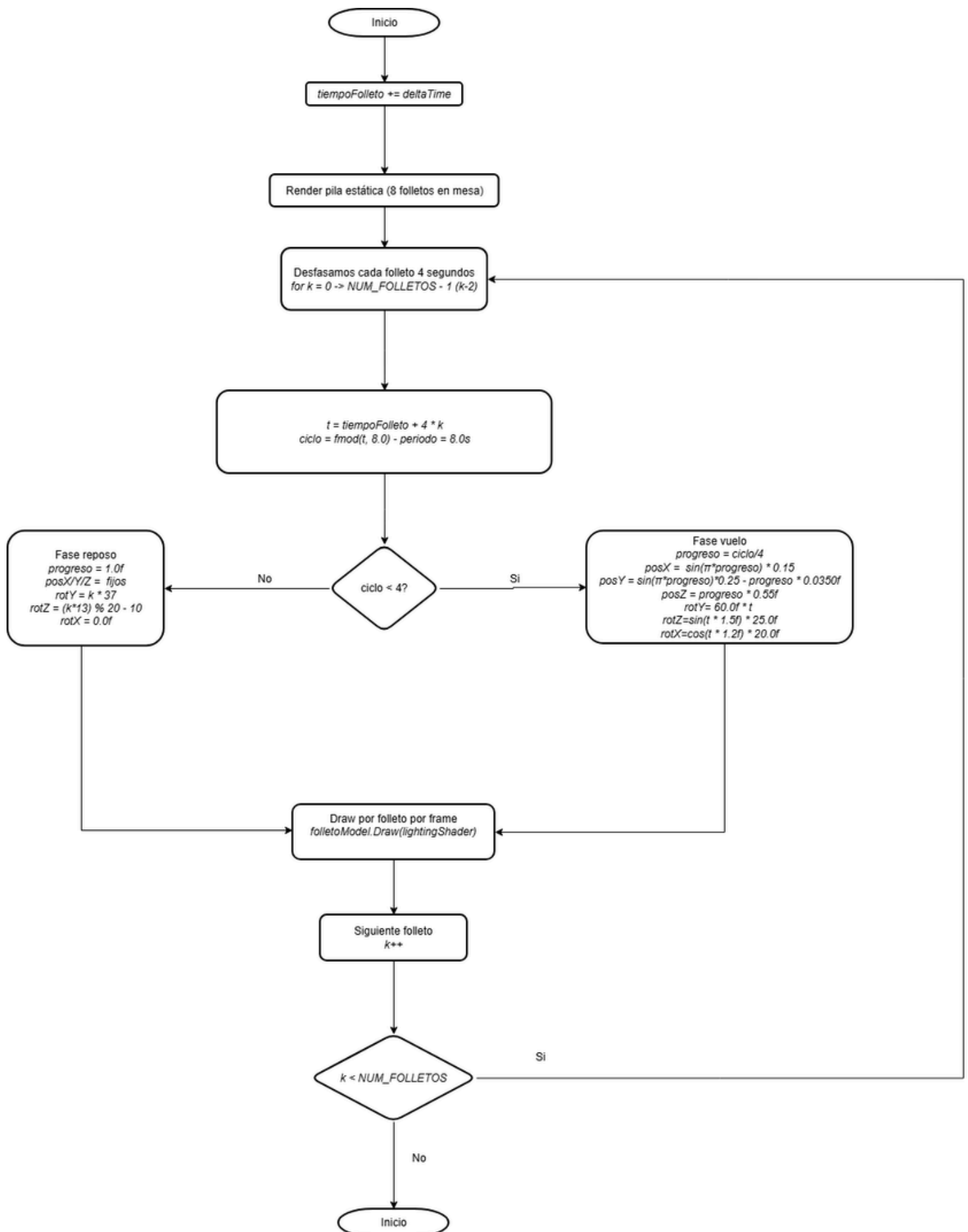
6.6 Diagrama de la animación del brazo robótico



6.7 Diagrama de la animación de la cámara de seguridad



6.8 Diagrama de la animación de los folletos



7. Documentación del código

La presente sección describe los componentes técnicos más relevantes del sistema desarrollado, explicando el funcionamiento interno del código fuente, los algoritmos implementados y las decisiones de diseño tomadas durante el desarrollo. La documentación se organiza en tres grandes bloques: la inicialización del sistema, el sistema de iluminación basado en el modelo de Phong, y el sistema dual de skyboxes con cambio dinámico día/noche. En la siguiente sección se documentan a detalle las siete animaciones independientes que componen el proyecto.

Inicialización del sistema

Objetivo

Antes de entrar al ciclo principal de renderizado (game loop), el sistema realiza una secuencia de configuración que prepara el contexto de OpenGL, carga todos los recursos visuales del entorno y define los estados iniciales de las animaciones.

Configuración del contexto OpenGL

La inicialización comienza con la creación de la ventana mediante GLFW y la configuración del contexto gráfico:

```
glfwInit();
GLFWwindow* window = glfwCreateWindow(WIDTH, HEIGHT, "ProyectoFinal-Samuel Medina Villa-Eric
Ramirez", nullptr, nullptr);
glfwMakeContextCurrent(window);
glewExperimental = GL_TRUE;
glewInit();
glViewport(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT);
```

Una vez inicializado el contexto, se registran los callbacks de teclado y ratón que permiten al usuario interactuar con la cámara sintética y activar las animaciones controlables mediante teclas dedicadas.

Carga de modelos y shaders

Tras la configuración del contexto, el sistema carga la totalidad de los modelos tridimensionales y los programas de shader necesarios para el renderizado. La carga se realiza una sola vez al inicio del programa, evitando recargas durante el ciclo de renderizado:

```
Shader lightingShader("Shader/lighting.vs", "Shader/lighting.frag");
Shader lampShader("Shader/lamp.vs", "Shader/lamp.frag");
Shader emissiveShader("Shader/emissive.vs", "Shader/emissive.frag"); //sol y luna
Shader skyboxshader("Shader/skybox.vs", "Shader/skybox.frag");
Shader shaderAnimacion("Shader/anim.vs", "Shader/anim.frag");
```

```

Model Puente((char*)"Models/Puente/Puente.obj");
Model mamp1((char*)"Models/stands_2/mamp_1.obj"); .....

//stands de empresas
Model stand_pag((char*)"Models/stands_1/pg.obj");
Model stand_amazon((char*)"Models/stands_1/amazon.obj");
Model stand_oracle((char*)"Models/stands_1/oracle.obj");

//stands de asociaciones
Model stand_crofi((char*)"Models/stands_1/crofi.obj"); .....

```

Inicialización de keyframes

Las animaciones basadas en keyframes (visitante y brazo robótico) requieren que sus arreglos de fotogramas clave sean definidos antes de entrar al game loop. Esta inicialización se realiza mediante la función `resetElements()` y la primera llamada a `interpolation()`, que calcula los incrementos por paso entre el primer y segundo keyframe:

```

visResetElements();
visPlay = true;
visPlayIndex = 0;
vis_i_curr_steps = 0;
visInterpolation();

```

Una vez completada la inicialización, el sistema entra al ciclo principal de renderizado, que se ejecuta continuamente hasta que el usuario cierra la ventana.

Sistema de iluminación

Objetivo

Proporciona realismo visual al entorno mediante la simulación física aproximada del comportamiento de la luz sobre las superficies de los modelos, y permite generar dos atmósferas completamente distintas día y noche mediante un cambio dinámico en tiempo real.

El sistema implementa el modelo de reflexión de Phong, calculado por fragmento sobre todos los modelos estáticos del entorno mediante el shader `lighting.frag`. El modelo combina simultáneamente una luz direccional global, cuatro luces puntuales y veinticuatro reflectores cónicos

Luz direccional (sol/luna)

Se utiliza para representar el Sol durante el día y la Luna durante la noche. Su comportamiento se modifica en tiempo real según el valor de la bandera `modoDia`:

```

if (modoDia) {
    //SOL
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.direction"), 1.3f, -1.1f, 1.0f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.ambient"), 0.35f, 0.32f, 0.28f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.diffuse"), 0.80f, 0.76f, 0.68f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.specular"), 0.45f, 0.45f, 0.40f);
}
else {
    //LUNA
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.direction"), -1.3f, -1.1f, 1.0f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.ambient"), 0.08f, 0.10f, 0.18f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.diffuse"), 0.20f, 0.25f, 0.40f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.specular"), 0.30f, 0.32f, 0.45f);
}

```

Reflectores sobre los stands (24 spotlights)

Cada uno de los seis stands del puente posee cuatro lámparas tipo spotlight que iluminan de forma focalizada sus elementos promocionales. El total de veinticuatro reflectores cónicos se gestiona mediante un arreglo en el fragment shader (standSpots[0..23]) cuyas posiciones, direcciones y propiedades se configuran individualmente

La configuración de cada spotlight se realiza dentro de un bucle for que asigna sus parámetros uniformes en función del stand al que pertenece, para fines prácticos se pone el ejemplo de un stand

```

for (int i = 0; i < 4; i++) {
    std::string base = "standSpots[" + std::to_string(i) + "]";

    //posicion
    glUniform3f(glGetUniformLocation(lightningShader.Program, (base + ".position").c_str()),
        oracleSpots[i].x, oracleSpots[i].y, oracleSpots[i].z);
    glUniform3f(glGetUniformLocation(lightningShader.Program, (base + ".direction").c_str()),
        0.0f, -1.0f, 0.0f);

    //angulos de apertura
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base + ".cutOff").c_str()),
        glm::cos(glm::radians(8.0f)));
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base + ".outerCutOff").c_str()),
        glm::cos(glm::radians(30.0f)));

    //color de la luz: blanco
    glUniform3f(glGetUniformLocation(lightningShader.Program, (base + ".ambient").c_str()),
        0.0f, 0.0f, 0.0f); //sin ambiente para evitar que ilumine fuera del cono y dar efecto exagerado del
spotlight
    glUniform3f(glGetUniformLocation(lightningShader.Program, (base + ".diffuse").c_str()),
        1.0f * spot_intensity, 1.0f * spot_intensity, 1.0f * spot_intensity);
    glUniform3f(glGetUniformLocation(lightningShader.Program, (base + ".specular").c_str()),
        1.0f * spot_intensity, 1.0f * spot_intensity, 1.0f * spot_intensity);

    //atenuacion
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base + ".constant").c_str()), 1.0f);
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base + ".linear").c_str()), 5.0f);
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base + ".quadratic").c_str()), 50.0f);
}

```

Luces puntuales del techo (4 point lights)

Las cuatro lámparas distribuidas a lo largo del techo del puente se modelan como luces puntuales que se activan únicamente durante el modo noche. Estas luces utilizan una tabla de atenuación calibrada para una distancia de aproximadamente 100 unidades, con valores específicos para los coeficientes constante, lineal y cuadrático:

```
glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[0].constant"), 1.0f);
glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[0].linear"), 0.045f); //usamos
los valores de linear y quadratic para distance 100
//dados en la tabla de pointLights vista en clase de teoria (grupo 05)
glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[0].quadratic"), 0.0075f);
```

Sistema de skyboxes

Objetivo

El sistema de skyboxes proporciona el entorno visual que rodea al usuario. Se implementó un sistema dual compuesto por dos skyboxes simultáneos: uno interior que se ajusta al volumen del puente y otro exterior que rodea toda la estructura del puente. Cada uno cuenta con dos versiones, una diurna y otra nocturna, que se intercambian dinámicamente mediante la tecla L.

Estructura de almacenamiento

El proyecto gestiona cuatro cubemaps independientes, cada uno compuesto por seis texturas que representan las caras de un cubo (derecha, izquierda, arriba, abajo, frente, atrás). Las texturas se cargan en memoria de video mediante la función LoadCubemap de la clase utilitaria TextureLoading:

```
GLuint cubemapTexture          = TextureLoading::LoadCubemap(faces);
GLuint cubemapTextureExterior  = TextureLoading::LoadCubemap(facesExterior);
GLuint cubemapTextureNoche     = TextureLoading::LoadCubemap(facesNoche);
GLuint cubemapTextureExteriorNoche =TextureLoading::LoadCubemap(facesExteriorNoche);
```

Skybox exterior

El skybox exterior se renderiza al inicio del game loop, antes de cualquier modelo opaco. Se deshabilita el depth mask durante su renderizado, lo que evita que el skybox bloquee la visibilidad de los modelos posteriores:

```
glDepthFunc(GL_EQUAL);
glDepthMask(GL_FALSE);
skyboxshader.Use();
// ... configuración de uniformes ...
glDrawArrays(GL_TRIANGLES, 0, 36);
glDepthMask(GL_TRUE);
glDepthFunc(GL_LESS);
```

Skybox interior

A diferencia del skybox exterior, el interior debe ajustarse exactamente al volumen del puente para que el cielo visible a través de las aberturas laterales coincida con la perspectiva real del entorno. Para ello se aplica una matriz de modelo personalizada que posiciona y escala el cubo del skybox interior según las dimensiones del puente, obtenidas directamente de los datos de vértices del modelo en Blender:

```
model = glm::mat4(1);  
model = glm::translate(model, glm::vec3(-0.0413f, 0.6800f, -2.9475f));  
model = glm::scale(model, glm::vec3(1.4619f, 0.1850f, 0.4875f));
```

Cambio dinámico día/noche

El cambio entre el modo día y el modo noche se controla mediante la tecla L, que invierte el valor de la bandera booleana modoDia. En cada iteración del game loop, esta variable determina qué cubemap se enlaza al momento de hacer el draw call mediante una condición ternaria:

```
glBindTexture(GL_TEXTURE_CUBE_MAP, modoDia ? cubemapTexture : cubemapTextureNoche);
```

El mismo patrón se aplica al skybox exterior con sus respectivas variantes.

Animación de la cámara de seguridad

Objetivo

Sobre uno de los pilares del puente se ubica una cámara de seguridad compuesta por dos modelos separados: una base fija anclada al pilar y una cabeza giratoria. La animación reproduce el movimiento de paneo automático característico de los sistemas de vigilancia, generando un movimiento oscilatorio continuo de la cabeza mientras la base permanece estática.

El objetivo de separar el modelo en dos partes es permitir que la rotación se aplique únicamente sobre el componente giratorio, manteniendo intacta la base que está fijada al pilar. Esta separación se realizó previamente en Blender exportando los dos componentes como modelos OBJ independientes.

Fundamento matemático

El movimiento de paneo se modela mediante una función sinusoidal continua aplicada al ángulo de rotación de la cabeza:

$$\text{rotCamera}(t) = 45^\circ \cdot \sin(t \cdot \pi/10)$$

donde t representa el tiempo acumulado desde el inicio del programa, expresado en segundos. La función sinusoidal produce un movimiento de oscilación natural con las siguientes propiedades:

Amplitud: $\pm 45^\circ$ respecto al centro, lo que produce un rango total de paneo de 90° .

Frecuencia angular: $\pi/10$ rad/s.

Periodo: 20 segundos por ciclo completo (ida y vuelta).

Implementación

La actualización del ángulo se realiza en cada iteración del game loop dentro de la función Animation():

```
tiempoCamera += deltaTime;  
rotCamera = 45.0f * sin(tiempoCamera * (glm::pi<float>() / 10.0f));
```

La rotación se aplica exclusivamente sobre el modelo camCabeza utilizando como pivote el centro físico de la cabeza

```
modelCamCabeza = glm::translate(modelCamCabeza, centroCamCabeza * 0.005f);  
modelCamCabeza = glm::rotate(modelCamCabeza, glm::radians(rotCamera), glm::vec3(0.0f, 1.0f, 0.0f));  
modelCamCabeza = glm::translate(modelCamCabeza, -centroCamCabeza * 0.005f);  
modelCamCabeza = glm::scale(modelCamCabeza, glm::vec3(0.005f, 0.005f, 0.005f));
```

Control de reproducción

Esta animación se ejecuta de manera automática al iniciar el programa y se mantiene activa de forma indefinida durante todo el ciclo de vida de la aplicación. No requiere intervención del usuario ni puede pausarse, dado que reproduce el comportamiento esperado de un sistema de vigilancia que opera continuamente.

Animación de rotación de logos

Objetivo

Sobre cada uno de los tres stands de empresas patrocinadoras (Oracle, Amazon y P&G) flota un logotipo tridimensional independiente que puede rotar sobre su eje vertical. La animación implementa una rotación procedural acumulativa que permite al usuario activar o desactivar cada logotipo de forma independiente mediante teclas numéricas, generando un efecto visual atractivo similar al de las exhibiciones promocionales reales.

Fundamento técnico

La rotación se construye mediante una transformación incremental aplicada en cada fotograma sobre la matriz de modelo de cada logo. La velocidad angular es constante, y el ángulo acumulado se almacena en un acumulador de tiempo independiente para cada logo:

```
if (playOracle) tiempoOracle += deltaTime;  
if (playAmazon) tiempoAmazon += deltaTime;  
if (playPG) tiempoPG += deltaTime;
```

La transformación aplicada utiliza el centro físico del logotipo como pivote, extraído de la escena en Blender, garantizando que la rotación ocurra sobre el eje vertical local del modelo y no sobre el origen global de coordenadas

Ejemplo con el stand de Oracle, es el mismo funcionamiento para los otros dos logos

```
glm::vec3 centroOracle(-255.8f, 136.83f, -48.15f);

modelOracle = glm::translate(modelOracle, centroOracle);
if (playOracle)
    modelOracle = glm::rotate(modelOracle,
        tiempoOracle, glm::vec3(0.0f, 1.0f, 0.0f));
modelOracle = glm::translate(modelOracle, -centroOracle);
```

Control de reproducción

Cada logo posee su propia bandera de activación independiente y se controla mediante una tecla numérica dedicada:

Tecla 1: activa o desactiva la rotación del logotipo de Oracle.

Tecla 2: activa o desactiva la rotación del logotipo de Amazon.

Tecla 3: activa o desactiva la rotación del logotipo de P&G

Al desactivar cualquiera de las rotaciones, el acumulador de tiempo correspondiente se reinicia a cero, lo cual regresa al logotipo a su orientación original

```
if (key == GLFW_KEY_1 && action == GLFW_PRESS) {
    playOracle = !playOracle;
    if (!playOracle) tiempoOracle = 0.0f;
}
```

Animación de los folletos volando

Objetivo

Simula el comportamiento típico de los folletos promocionales en un evento académico: una pila de folletos descansa sobre la mesa del stand de P&G, y cada cierto tiempo un par de folletos vuela desde la pila hacia el piso del puente, donde finalmente se asienta.

Esta animación es la más compleja del proyecto desde el punto de vista matemático porque combina tres elementos: una trayectoria curva en el espacio tridimensional generada mediante funciones trigonométricas, una lógica de dos fases (vuelo y reposo) controlada por una función módulo, y rotaciones animadas durante el vuelo que se congelan al aterrizar.

Fundamento matemático

Cada folleto sigue un ciclo de ocho segundos compuesto por dos fases. El progreso temporal dentro del ciclo se calcula utilizando la función módulo:

$$\text{ciclo} = \text{fmod}(t + k \cdot 4, 8)$$

donde t es el tiempo global de la animación, k es el índice del folleto (0 o 1) y la suma de $k \cdot 4$ produce un desfase de cuatro segundos entre los folletos, evitando que vuelen en sincronía. El valor del progreso normalizado dentro de cada fase se calcula como:

$$\begin{aligned} \text{progreso} &= \text{ciclo} / 4 && (\text{si ciclo} < 4, \text{ fase de vuelo}) \\ \text{progreso} &= 1 && (\text{si ciclo} \geq 4, \text{ fase de reposo}) \end{aligned}$$

La trayectoria del folleto durante la fase de vuelo se modela combinando funciones sinusoidales y desplazamiento lineal

```
float posX = pilaPosX + sin(progreso * 3.1416f) * 0.15f;
float posY = pilaPosY + sin(progreso * 3.1416f) * 0.25f - progreso * 0.0350f;
float posZ = pilaPosZ + progreso * 0.55f;
```

Las rotaciones durante el vuelo se calculan también mediante funciones trigonométricas independientes para cada eje:

```
rotY = t * 60.0f;
rotZ = sin(t * 1.5f) * 25.0f;
rotX = cos(t * 1.2f) * 20.0f;
```

Lógica de dos fases

El comportamiento del folleto cambia radicalmente al pasar de la fase de vuelo a la fase de reposo. Esta lógica se implementa mediante una bandera booleana `enReposo` que altera tanto la trayectoria como las rotaciones:

```
float ciclo = fmod(t, 8.0f);
float progreso;
bool enReposo = false;

if (ciclo < fasevuelo) {
    progreso = ciclo / fasevuelo;
} else {
    progreso = 1.0f;
    enReposo = true;
}
```

Durante la fase de reposo, las rotaciones del folleto se congelan en una pose final simulando que el folleto ha caído al piso con una orientación natural. Cada folleto cae con una orientación ligeramente distinta calculada a partir de su índice, lo que evita que ambos folletos terminen exactamente en la misma posición:

```
if (enReposo) {
    rotY = (float)(k * 37);
    rotZ = (float)((k * 13) % 20 - 10);
    rotX = 0.0f;
} else {
    rotY = t * 60.0f;
    rotZ = sin(t * 1.5f) * 25.0f;
    rotX = cos(t * 1.2f) * 20.0f;
}
```

Pila estática de folletos

Complementando la animación de los folletos volando, sobre la mesa del stand de P&G se renderiza una pila estática de folletos apilados con ligeras variaciones de posición y rotación lo que produce una apariencia natural de papeles acumulados:

```
for (int k = 0; k < 8; k++) {
    glm::mat4 modelPila = glm::mat4(1.0f);
    modelPila = glm::translate(modelPila, glm::vec3(
        pilaPosX + (k % 3) * 0.015f - 0.015f,
        pilaPosY + k * 0.001f,
        pilaPosZ + ((k * 7) % 5) * 0.008f - 0.02f));
    modelPila = glm::rotate(modelPila,
        glm::radians((float)(k * 13)), glm::vec3(0.0f, 1.0f, 0.0f));
    folletoModel.Draw(lightningShader);
}
```

Las expresiones $(k \% 3)$ y $(k * 7) \% 5$ generan patrones pseudoaleatorios sin necesidad de utilizar generadores de números aleatorios reales

Control de reproducción

La animación se ejecuta automáticamente al iniciar el programa. La tecla F permite reiniciar el ciclo del acumulador tiempoFolleto, lo que es útil para reposicionar los folletos en sus orígenes en caso de querer observar nuevamente el inicio del vuelo:

```
if (key == GLFW_KEY_F && action == GLFW_PRESS) {
    tiempoFolleto = 0.0f;
}
```

Animación del visitante caminando

Objetivo

El visitante es un personaje animado que recorre la totalidad del puente de manera continua. La animación combina dos técnicas distintas: animación por KeyFrames con interpolación lineal para controlar el desplazamiento y la rotación globales del personaje, y animación esquelética por huesos para reproducir el ciclo de caminata sobre el modelo FBX cargado desde Mixamo.

El objetivo de combinar ambas técnicas es lograr un personaje que se mueva por el espacio siguiendo una trayectoria predefinida (controlada por keyframes) mientras realiza un ciclo de caminata realista. Esta combinación reproduce el comportamiento natural de un visitante que se desplace observando los stands del puente.

Estructura de almacenamiento

Los fotogramas clave del recorrido se almacenan en un arreglo de estructuras VIS_FRAME. Cada estructura define una pose espacial del visitante en un instante determinado, junto con el número de pasos de interpolación que separan ese keyframe del siguiente:

```
typedef struct _vis_frame {
    float visPosX;      float visPosXInc;
    float visRotY;      float visRotYInc;
    int maxSteps;
} VIS_FRAME;
```

Definición del recorrido

Se definieron diez keyframes (VIS_MAX_FRAMES = 10) que describen el ciclo completo del visitante a lo largo del puente. El recorrido sigue la siguiente secuencia:

Parte del centro y camina hacia el extremo derecho del puente.

Gira 90° para observar uno de los stands del lado derecho.

Tras una pausa, gira otros 90° y camina hacia el extremo opuesto.

Repite el patrón de pausa y giro frente a un stand del lado izquierdo.

Camina de regreso al centro y reinicia el ciclo de forma indefinida

Los valores específicos de los keyframes se definen en la inicialización del programa:

```
VisKF[0] = { 0.0f, 0, 0.0f, 0, 800 };
VisKF[1] = { 240.0f, 0, 0.0f, 0, 50 };
VisKF[2] = { 240.0f, 0, 90.0f, 0, 200 };
VisKF[3] = { 240.0f, 0, 90.0f, 0, 50 };
VisKF[4] = { 240.0f, 0, 180.0f, 0, 1600 };
// ... continúa hasta el keyframe 9
```

Interpolación lineal

Para producir un movimiento suave entre keyframes, se aplica interpolación lineal calculando los incrementos por paso. La ecuación general utilizada es:

$$\text{Incremento} = (\text{ValorFinal} - \text{ValorInicial}) / \text{NúmeroDePasos}$$

La función visInterpolation() calcula automáticamente estos incrementos cada vez que se completa un keyframe y se transita al siguiente:

```
VisKF[visPlayIndex].visPosXInc =(VisKF[visPlayIndex + 1].visPosX - VisKF[visPlayIndex].visPosX)/
vis_i_max_steps;
```

Lógica de doble estado

Mientras el visitante se desplaza o gira, se reproduce el ciclo de caminata utilizando un acumulador de tiempo (tiempoAnimVisitante). Cuando el visitante se encuentra en reposo frente a un stand, el modelo se congela en una pose fija que corresponde al fotograma exacto donde el personaje aparenta estar de pie observando:

```

bool hayDesplazamiento = (std::abs(VisKF[visPlayIndex].visPosXInc) > 0.0001f);
bool hayGiro = (std::abs(VisKF[visPlayIndex].visRotYInc) > 0.0001f);

if (hayDesplazamiento || hayGiro)
{
    tiempoAnimVisitante += deltaTime * 0.8f;
    visitante.Draw(shaderAnimacion, tiempoAnimVisitante);
}
else
{
    visitante.Draw(shaderAnimacion, poseFijaVisitante);
}

```

Animación del presentador del stand

Objetivo

En uno de los stands se encuentra un personaje articulado denominado presentador, cuya función dentro del entorno es dar la bienvenida a los visitantes que se acercan a su zona. La animación se construye mediante una jerarquía de cuatro partes (cuerpo, cabeza, antebrazo y mano) cuyas rotaciones se generan a partir de funciones sinusoidales con frecuencias y amplitudes distintas para cada articulación.

Fundamento matemático

La rotación de cada articulación se calcula en función del tiempo acumulado (expositorTiempo), aplicando la ecuación general:

$$\text{rotación}(t) = \text{amplitud} \cdot \sin(t \cdot \text{frecuencia})$$

donde t es el tiempo acumulado en segundos desde el inicio de la animación, la amplitud determina el rango angular del movimiento expresado en grados, y la frecuencia determina la velocidad angular en radianes por segundo. Los parámetros específicos asignados a cada articulación son los siguientes:

Antebrazo: amplitud 20° y frecuencia 5.0 rad/s, sobre el eje X.

Mano: amplitud 25° y frecuencia 5.0 rad/s, sobre el eje Y.

Cabeza: amplitud 15° y frecuencia 1.5 rad/s, sobre el eje Y.

Implementación

Las rotaciones se calculan en cada iteración del game loop dentro de la función Animation() siempre que la bandera animExpositor esté activa:

```

if (animExpositor)
{
    expositorTiempo += deltaTime;
    expositorRotMano = 25.0f * sin(expositorTiempo * 5.0f);
    expositorRotCabeza = 15.0f * sin(expositorTiempo * 1.5f);
    expositorRotAntebrazo = 20.0f * sin(expositorTiempo * 5.0f);
}

```

Las rotaciones calculadas se aplican posteriormente sobre las matrices de transformación de cada parte del cuerpo, respetando la jerarquía padre-hijo: el cuerpo es el nodo raíz, la cabeza y el antebrazo son hijos directos del cuerpo, y la mano es hija del antebrazo. Esto permite que la mano herede automáticamente las transformaciones del antebrazo, reproduciendo el comportamiento anatómico correcto:

```
glm::mat4 antebrazoMat = modelPres;
antebrazoMat = glm::translate(antebrazoMat, glm::vec3(-16.23f, 124.96f, -13.10f));
antebrazoMat=glm::rotate(antebrazoMat,glm::radians(expositorRotAntebrazo), glm::vec3(1.0f, 0.0f, 0.0f));
antebrazoMat = glm::translate(antebrazoMat, glm::vec3(16.23f, -124.96f, 13.10f));

glm::mat4 manoMat = antebrazoMat;
// La mano hereda la transformación del antebrazo
```

Los pivotes de rotación se obtuvieron directamente del modelo en Blender.

Control de reproducción

La animación se activa o desactiva mediante la tecla E. Cuando se desactiva, todas las articulaciones regresan suavemente a su posición inicial al reiniciarse el acumulador `expositorTiempo`:

```
if (keys[GLFW_KEY_E])
{
    animExpositor = !animExpositor;
    if (!animExpositor)
    {
        expositorRotMano = 0.0f;
        expositorRotCabeza = 0.0f;
        expositorTiempo = 0.0f;
    }
}
```

Animación del brazo robótico

Objetivo

Sobre la mesa de uno de los stands se encuentra un brazo robótico articulado. La animación reproduce el ciclo completo de manipulación industrial: el brazo toma una pieza imaginaria, la levanta, la rota hacia el lado opuesto, la deposita y regresa a su posición de reposo. La técnica empleada es KeyFrames jerárquicos, que combina la animación por fotogramas clave con una estructura de transformaciones encadenadas padre-hijo.

Estructura de almacenamiento

Cada fotograma clave almacena las rotaciones de las cuatro articulaciones del brazo, junto con sus respectivos incrementos por paso y el número total de pasos hasta el siguiente keyframe:

```
typedef struct _brazo_frame {
    float baseRot;      float baseRotInc;
    float hombroRot;    float hombroRotInc;
    float codoRot;      float codoRotInc;
    float garraRot;     float garraRotInc;
    int maxSteps;
} BRAZO_FRAME;
```

Jerarquía cinemática

El brazo se compone de cinco elementos ordenados en una jerarquía padre-hijo:

base fija → pedestal giratorio → hombro → brazo/codo → garra

La técnica de rotación sobre pivote se aplica mediante la composición de tres transformaciones: traslación al punto de pivote, rotación, y traslación inversa para regresar al sistema de coordenadas original:

```
glm::mat4 mPedestal = modelBrazo;
mPedestal = glm::translate(mPedestal, pivotePedestal);
mPedestal = glm::rotate(mPedestal, glm::radians(brazo_baseRot), glm::vec3(0.0f, 1.0f, 0.0f));
mPedestal = glm::translate(mPedestal, -pivotePedestal);
```

Los pivotes de cada articulación (pedestal, hombro, codo y garra) se extrajeron directamente del modelo en Blender

Definición del ciclo

Se definieron nueve fotogramas clave (BRAZO_MAX_FRAMES = 9) que describen el ciclo completo de manipulación:

```
BrazoKF[0] = { 0.0f, 0, 0.0f, 0, 0.0f, 0, 0.0f, 0, 100 }; // reposo
BrazoKF[1] = { 45.0f, 0, 0.0f, 0, 0.0f, 0, 0.0f, 0, 120 }; // gira base
BrazoKF[2] = { 45.0f, 0, 30.0f, 0, 10.0f, 0, 5.0f, 0, 150 }; // baja
BrazoKF[3] = { 45.0f, 0, 30.0f, 0, 10.0f, 0, -5.0f, 0, 60 }; // balanceo
BrazoKF[4] = { 45.0f, 0, -10.0f, 0, -30.0f, 0, 0.0f, 0, 120 }; // levanta
BrazoKF[5] = { -45.0f, 0, -10.0f, 0, -30.0f, 0, 5.0f, 0, 180 }; // gira
BrazoKF[6] = { -45.0f, 0, 28.0f, 0, 8.0f, 0, 5.0f, 0, 150 }; // baja
BrazoKF[7] = { -45.0f, 0, 0.0f, 0, 0.0f, 0, 0.0f, 0, 120 };
BrazoKF[8] = { 0.0f, 0, 0.0f, 0, 0.0f, 0, 0.0f, 0, 1 };
```

Reproducción de la animación

Durante cada iteración del game loop, las rotaciones de las articulaciones se actualizan sumando los incrementos calculados previamente por la función `brazoInterpolation()`. Cuando se alcanza el número máximo de pasos del keyframe actual, el sistema avanza al siguiente keyframe y recalcula los incrementos:


```

if (brazoPlay) {
    if (brazo_i_curr_steps >= brazo_i_max_steps) {
        brazoPlayIndex++;
        if (brazoPlayIndex >= BRAZO_MAX_FRAMES - 1) {
            brazoPlayIndex = 0;
            brazo_i_curr_steps = 0;
            brazoResetElements();
            brazoInterpolation();
        }
        else {
            brazo_i_curr_steps = 0;
            brazoInterpolation();
        }
    }
    else {
        brazo_baseRot += BrazoKF[brazoPlayIndex].baseRotInc;
        brazo_hombroRot += BrazoKF[brazoPlayIndex].hombroRotInc;
        brazo_codoRot += BrazoKF[brazoPlayIndex].codoRotInc;
        brazo_garraRot += BrazoKF[brazoPlayIndex].garraRotInc;
        brazo_i_curr_steps++;
    }
}
}

```

Control de reproducción

La animación inicia automáticamente al ejecutar el programa, se reproduce en bucle indefinido y puede pausarse o reanudarse mediante la tecla B. Al pausarse, el brazo conserva la posición exacta del fotograma actual; al reanudarse, continúa desde ese punto sin saltos visuales

Animación del perro robot

Objetivo

En el stand de CROFI se exhibe un modelo de perro robot animado mediante la técnica de animación esquelética. El perro robot fue trabajado por completo de forma manual: el rigging se construyó vértice por vértice en Blender, y la animación se generó mediante keyframes en la línea de tiempo del software, lo que permitió diseñar un movimiento específico para el contexto del stand.

Fundamento técnico

La animación esquelética permite deformar la malla de un modelo tridimensional en función de las transformaciones de un esqueleto interno (huesos), donde cada vértice se asocia a uno o más huesos mediante pesos de influencia. La posición final de cada vértice durante el renderizado se calcula como una suma ponderada de las transformaciones de los huesos que lo afectan:

$$\text{posición_final} = \sum (\text{peso_i} \cdot \text{matriz_hueso_i} \cdot \text{posición_local})$$

donde la suma se realiza sobre todos los huesos i que influyen al vértice, el peso_i es el factor de influencia de cada hueso, la matriz_hueso_i es la transformación acumulada del hueso en el instante actual, y la posición_local es la coordenada original del vértice en el sistema de referencia de la malla en reposo

Proceso de implementación

El proceso completo de implementación se desarrolló en cinco etapas:

Modelado y rigging en Blender: se construyó manualmente el esqueleto del perro robot, definiendo huesos para las cuatro patas, la cadera, el torso y la cabeza.

Asignación de pesos: cada vértice de la malla se vinculó a uno o más huesos con su respectivo peso de influencia, ajustando manualmente las zonas de transición entre huesos para evitar deformaciones bruscas.

Creación de animación por fotogramas en Blender: se generó una animación de saludo mediante rotaciones de los huesos sobre la línea de tiempo.

Exportación en formato FBX: el archivo final incluye la malla, el esqueleto, los pesos y la animación horneada.

Carga en OpenGL mediante Assimp: la clase `ModeloAnimado` procesa la jerarquía de huesos, los canales de animación y los pesos por vértice, y los envía al shader `anim.vs` para su procesamiento en GPU.

Reproducción de la animación

La animación se controla mediante un acumulador de tiempo y una bandera de estado. Cuando la bandera `playPerro` está activa, el acumulador avanza con el tiempo del fotograma actual y se pasa como parámetro al método `Draw()` del modelo:

```
if (playPerro) {
    tiempoPerro += deltaTime;
}
perroRobot.Draw(shaderAnimacion, tiempoPerro);
```

Internamente, la clase `ModeloAnimado` utiliza el tiempo recibido para interpolar entre los keyframes de la animación cargada desde el FBX, calculando la matriz de transformación de cada hueso en el instante exacto y enviándola al vertex shader como parte del arreglo de uniforms de huesos

Control de reproducción

Por defecto la animación inicia pausada. Al presionar la tecla `P` por primera vez, el perro comienza a moverse; al volver a presionarla, se detiene y regresa al fotograma cero del ciclo:

```
if (key == GLFW_KEY_P && action == GLFW_PRESS) {
    playPerro = !playPerro;
    if (!playPerro) tiempoPerro = 0.0f;
}
```

8. Análisis de Costos

El presente análisis cuantifica los recursos económicos invertidos en el desarrollo de la parte individual del entorno virtual interactivo, así como el precio de venta estimado de mi contribución si se ofreciera como servicio profesional a un cliente externo. Todos los datos económicos están respaldados por fuentes verificables del mercado mexicano consultadas en mayo de 2026, listadas en la parte del bibliografía

8.1 Costo de desarrollo

Mi rol dentro del proyecto corresponde al de desarrollador junior en computación gráfica, perfil que combina programación en C++ con OpenGL, modelado 3D en Blender, manejo de shaders y animación . Se establece una tarifa de \$200.00 MXN/hr, valor que se ubica en el rango inferior reportado por la industria mexicana para desarrolladores junior (entre \$150 y \$300 pesos mexicanos por hora para desarrolladores junior en México), considerando que el perfil de desarrollo gráfico tridimensional exige conocimientos especializados adicionales a los de un desarrollador junior genérico.

La estimación de horas se realiza a partir del registro de mis commits en el repositorio de GitHub y la duración real del proyecto (del 19 de abril al 26 de mayo de 2026, equivalente a 5.5 semanas). Se considera una dedicación efectiva de aproximadamente 22 horas semanales, que contabiliza no solo el tiempo neto de codificación sino también las horas invertidas en pruebas iterativas, depuración de errores, ajuste de texturas, calibración de pivotes en Blender y validación visual de las animaciones.

Rol técnico	Horas semanales	Semanas	Horas totales	Tarifa (MXN/hr)	Subtotal (MXN)
Modelado del puente, sistema de skyboxes con cambio día/noche, iluminación de stands (24 spotlights) y cuatro animaciones (visitante, presentador, brazo robótico, folletos)	22	5.5	120	200	24000
Total mano de obra			120		24,000

8.2 Costo de software

La siguiente tabla enumera cada herramienta utilizada con su licencia correspondiente y su costo comercial equivalente como referencia.

Software / Herramienta	Versión	Tipo de licencia	Costo real (MXN)	Costo comercial equivalente (MXN)
Visual Studio Community 2022	17.x	Gratuita para estudiantes y proyectos personales	\$0.00	\$3,480.00 (Professional)
Blender	4.x	GPL (Open Source)	\$0.00	\$0.00
OpenGL 3.3 Core Profile	3.3	Especificación abierta de Khronos Group	\$0.00	\$0.00
GLEW (OpenGL Extension Wrangler)	2.1.0	Modified BSD / MIT	\$0.00	\$0.00
GLFW (Graphics Library Framework)	3.3	zlib/libpng	\$0.00	\$0.00
GLM (OpenGL Mathematics)	0.9.9	MIT	\$0.00	\$0.00
SOIL2 (Simple OpenGL Image Library)	1.x	MIT-0	\$0.00	\$0.00
Assimp (Open Asset Import Library)	5.x	BSD 3-Clause	\$0.00	\$0.00
Mixamo (animaciones esqueléticas)	Web	Gratuita con cuenta Adobe	\$0.00	\$0.00
Git / GitHub	2.x	Gratuita para repositorios públicos	\$0.00	\$0.00
Total software (costo real)			\$0.00	\$3,480.00

8.3 Costo por hardware

Para el desarrollo y prueba de mi parte del proyecto utilicé un equipo de cómputo personal. El precio de referencia corresponde a una laptop gamer de gama media con GPU dedicada, ubicada en el rango medio del mercado mexicano (laptops gamer en México con RTX 4050 o superiores oscilan entre \$22,000 y \$36,000 MXN). Dado que este equipo cumple funciones adicionales fuera del proyecto, su costo se amortiza considerando una vida útil de 5 años y se atribuye únicamente la fracción correspondiente al tiempo de desarrollo (5.5 semanas de 260 semanas totales)

Concepto	Cantidad	Costo unitario (MXN)	Vida útil (semanas)	Semanas usadas	Costo imputable (MXN)
Laptop con GPU dedicada (gama media)	1	25000	260	5.5	528.85
Periféricos (mouse, teclado, monitor externo)	1 set	3500	260	5.5	74.04
Total hardware					602.89

8.4 Costos indirectos

Las tarifas de internet residencial corresponden al plan Telmex Infinitum básico (\$349 MXN mensuales por 80 Megas, escalando a los \$449 MXN por 150 Megas), tomando el promedio de \$399 MXN/mes. El consumo eléctrico se calcula sobre el promedio de tarifa doméstica de CFE para una laptop con cargador de 150W operando durante el desarrollo.

Concepto	Detalle	Costo (MXN)
Servicio de internet	399.00/mes x 1.4 meses (5.5 semanas)	558.6
Energía eléctrica	Consumo de equipo de cómputo durante 120 horas de uso efectivo	350
Consumibles de papelería	Impresiones, notas, bocetos en papel	75
Total costos indirectos		983.6

8.5 Resumen de costos

Rubro	Costo (MXN)	Porcentaje
Mano de obra	24000	93.50%
Software	0	0.00%
Hardware	602.89	2.35%
Costos indirectos	983.6	3.83%
Subtotal costo de desarrollo	25586.49	99.68%
Imprevistos (0.32%)	82.42	0.32%
TOTAL COSTO DE DESARROLLO	25,668.91	100.00%

Precio de venta del proyecto

El precio de venta se calcula a partir del costo total de desarrollo, agregando un margen de utilidad razonable y considerando los costos administrativos asociados a la entrega del producto como un servicio profesional. Este precio refleja el valor que un cliente externo (por ejemplo, una institución educativa, una empresa patrocinadora de eventos o una organización estudiantil) pagaría por recibir la parte del entorno virtual interactivo correspondiente a mi contribución.

Concepto	Cálculo	Monto (MXN)
Costo de desarrollo	Total de la sección del resumen de costos	25668.91
Gastos administrativos	Facturación, gestión de proyecto, comunicación con cliente	2566.89
Soporte técnico post-entrega (3 meses, 8%)	Corrección de bugs y ajustes menores	2053.51
Subtotal		30289.31
Margen de utilidad (40%)	Ganancia neta del desarrollo, estándar de la industria de servicios gráficos especializados	12115.72
Precio neto antes de impuestos		42405.03
IVA (16%)	Impuesto al valor agregado en México	6784.8
PRECIO DE VENTA FINAL		49,189.83

Justificación del precio

El precio de venta final de \$49,189.83 MXN se considera adecuado dentro del mercado mexicano de desarrollo de aplicaciones gráficas interactivas tridimensionales. Para dimensionar su valor, se comparan a continuación cuatro alternativas de mercado equivalentes en distintos rangos de complejidad:

Alternativa de mercado	Costo aproximado (MXN)	Observaciones
Tour virtual 360° para página web (fotografía estática)	\$1,500 - \$7,000	Solo navegación entre fotos panorámicas, sin interactividad ni animaciones
Recorrido virtual residencial con VR	~\$25,000	Experiencia inmersiva pero sin lógica interactiva personalizada
Recorrido virtual residencial de alta complejidad	\$2,000 - \$185,000	Rango amplio que depende del nivel de detalle y animación
Aplicación interactiva en Unity con assets propios	\$50,000 - \$120,000	Aplicación 3D nativa con interactividad y animaciones complejas
Mi contribución al proyecto (OpenGL nativo)	\$49,189.83	Aplicación 3D nativa, 4 animaciones diferenciadas, sistema dual de skyboxes, modelado personalizado

Nota aclaratoria: El presente análisis de costos corresponde exclusivamente a mi contribución individual dentro del proyecto colaborativo, no al proyecto completo desarrollado en equipo. Las cifras aquí presentadas reflejan únicamente la inversión, los recursos y el valor comercial asociados a los entregables bajo mi responsabilidad. La parte correspondiente al otro integrante del equipo no se contabiliza en este documento.

9. Conclusiones

El desarrollo de este proyecto significó un reto técnico y de gestión considerable, pero también una oportunidad real para aplicar de forma integrada todos los conceptos vistos en el curso. . La construcción de un entorno virtual interactivo que reproduce fielmente el puente que conecta los edificios A y B de la Facultad de Ingeniería, junto con la implementación del modelo de iluminación de Phong, el sistema dual de skyboxes y las siete animaciones independientes, demostró que es posible combinar teoría y práctica en un producto funcional y visualmente coherente.

Mi contribución individual al proyecto se concentró en el modelado tridimensional del puente, su texturizado, el sistema de skyboxes con cambio dinámico día/noche, las veinticuatro luces tipo spotlight de los stands y cuatro de las siete animaciones del entorno: la del visitante caminante, la del presentador articulado, la del brazo robótico jerárquico y la de los folletos volando, cada una de estas tareas representó un aprendizaje técnico distinto.

Más allá de lo técnico, el proyecto reforzó la importancia de la gestión profesional del trabajo en equipo mediante el uso de GitHub.

Considero que el proyecto cumple con su propósito académico y demuestra el dominio integral de los conceptos del curso. Como aplicación práctica, el entorno virtual desarrollado funciona como un prototipo de tour virtual que podría utilizarse para planificar la distribución de stands en eventos universitarios reales, mostrar a empresas patrocinadoras cómo se vería su presencia en la facultad antes de un evento físico, y reducir los costos logísticos asociados a montajes presenciales. La curva de aprendizaje fue pronunciada, pero el resultado final justifica plenamente el esfuerzo invertido

10. Bibliografía

Blender Foundation. (s.f.). 5.1 — Blender. Blender.
<https://www.blender.org/download/releases/5-1/>

How to Use Mixamo with Blender: Full Beginner Guide. (s.f.). RebusFarm.
<https://rebusfarm.net/blog/how-to-use-mixamo-with-blender-full-beginner-guide>

Modelos descargados de internet Render People 3D models. (04 de enero de 2022). 3DTEAMZ. <https://3dteamz.com/render-people-3d-models/>

Industrial Robot Brazo Gratis 3D Modelos Stl - .stl descargar - Free3D. (s.f.). https://free3d.com/es/modelos-3d/stl-industrial-robot-brazo?dd_referrer=

Moon with 2k Textures | 3D model. (15 de diciembre de 2023). CGTrader.
<https://www.cgtrader.com/free-3d-models/space/planet/moon-with-2k-textures>

Robot Dog | 3D model. (05 de mayo de 2023). CGTrader. <https://www.cgtrader.com/free-3d-models/science/other/robot-dog-5cf6776b-9095-4e6b-8c79-26f2a05ef202>

Video camera | CGTrader. (s.f.). CGTrader.
<https://www.cgtrader.com/items/6117823/download-page>

Meshy. (13 de mayo de 2026). “Black Dome Pendant Light” - Descarga modelos 3D gratuitos generadas con Meshy AI por @dr.kanguro10. Meshy AI - El Generador De Modelos 3D Con IA Número 1. https://www.meshy.ai/es/3d-models/Black-Dome-Pendant-Light-v2-019e19eb-e4c7-7755-944d-51037603dd43?utm_medium=referral-program&utm_source=meshy&utm_content=T2AFGA&share_type=3d-models

Industrial Table Bar 120x30x112 | 3D model. (s.f.). CGTrader. <https://www.cgtrader.com/free-3d-models/interior/office-interior/industrial-table-bar-120x30x112-9d02243d-fe1c-46b6-864f-f602f1e0ca68>

Computrabajo México. (2026). Salario de Becario/a en México 2026. mx.computrabajo.com/salarios/becario

Glassdoor. (2026, marzo). Sueldo: Becario en desarrollo de software en México. glassdoor.com.mx/Sueldos/becario-en-desarrollo-de-software-sueldo-SRCH_KO0,33.htm

Accademia. (2024, 21 de diciembre). Costo por hora de desarrollo de software en México. accademia.es/costo-por-hora-de-desarrollo-de-software-en-mexico/

UNIR México. (2026, 13 de marzo). Desarrollador de software, qué hace y cuánto gana. mexico.unir.net/noticias/ingenieria/desarrollador-software/

Hardysystem. (2026). Visual Studio Professional 2026: IDE avanzado para desarrollo en Windows. hardysystem.com/productos/visual-studio-professional-2026-jpd81/

Microsoft Corporation. (2026). Comprar y descargar Visual Studio Professional 2026: Microsoft Store. microsoft.com/es-mx/d/visual-studio-professional-2026/dg7gmgf0vj96

CityShop. (2026, 11 de marzo). PC de Escritorio vs Laptop en 2026: Guía de compra. cityshop.mx/blog/pc-escritorio-vs-laptop-2026-guia-compra

Intercompras. (2026, 21 de abril). ¿Cuál es la mejor laptop gamer barata? intercompras.com/blog/mejor-laptop-gamer-barata/

Ámbito Financiero. (2026, 29 de enero). Cuál es internet más barato que conviene contratar en febrero 2026 en CDMX: precio de las tarifas de CFE, Starlink y Telmex. ambito.com/mexico/economia/cual-es-internet-mas-barato-que-conviene-contratar-febrero-2026-cdmx-precio-las-tarifas-cfe-starlink-y-telmex-n6239953

El Informador. (2026, 17 de marzo). Telmex, Izzi y CFE: ¿Cuál es el proveedor de internet más barato en México en 2026? informador.mx/mexico/telmex-izzi-y-cfe-cual-es-el-proveedor-de-internet-mas-barato-en-mexico-en-2026-20260317-0102.html

Habitissimo México. (2019, 11 de diciembre). Hacer recorrido virtual: Precio y cotizaciones. habitissimo.com.mx/presupuesto/hacer-recorrido-virtual

Cronoshare México. (2024, 3 de mayo). ¿Cuánto cuesta un Tour Virtual 360? Precios en 2026. cronoshare.com.mx/cuanto-cuesta/tour-virtual-360

Aulacm. (2026, 31 de enero). Cuánto cuesta una App móvil y cómo desarrollarla. Precio y tipos. aulacm.com/precio-desarrollar-app-aplicacion-movil/